

GeoJSON Görev Dosyası Ayrıştırıcı Rehberi

TEKNOFEST 2026 Robotaksi Yarışması

Bu doküman, yarışma günü verilen GeoJSON dosyalarını
anlaman, parse etmen ve robota aktarman için yazıldı.

Amaç: sadece yapmak değil, öğrenmek.

Nisan 2026

İçindekiler

Bölüm 1	Büyük Resim — Bu İş Nedir?
Bölüm 2	GeoJSON Nedir?
Bölüm 3	GPS Koordinatları ve Dönüşüm
Bölüm 4	Script Ne Yapıyor? (Parça Parça)
Bölüm 5	InitMissionAction ile Bağlantı
Bölüm 6	Test Etme Rehberi
Bölüm 7	Yarışma Günü Adım Adım
Bölüm 8	Script Kod Haritası
Bölüm 9	Sık Sorulan Sorular
Bölüm 10	Kaptana Sunum

Bölüm 1: Büyük Resim — Bu İş Nedir?

Yarışmada Ne Oluyor?

Robotaksi yarışmasında aracınız OTONOM olarak (insan müdahalesi olmadan) bir parkurda görevler yapıyor. Ama aracın NEREYE gideceğini bilmesi lazım.

İşte yarışma günü hakemler size bir dosya veriyor: GeoJSON dosyası. Bu dosya diyor ki: 'Araç şuradan başlayacak, şuraya yolcu alacak, şuraya yolcu bırakacak, en son şuraya park edecek.'

Problem Ne?

- 1) Bu dosya insan gözüyle okunması zor bir formatta (JSON) geliyor.
- 2) İçindeki koordinatlar GPS formatında (enlem/boylam derece cinsinden) ama robotun navigasyon sistemi (Nav2) METRE cinsinden x,y istiyor.

Senin Görevin

Bu dosyayı alıp:

- İnsan gözüyle okuyabilir hale getirmek (terminal özeti)
- Robotun anlayacağı formata çevirmek (YAML dosyası veya C++ kodu)

Bölüm 2: GeoJSON Nedir?

JSON Nedir Önce?

JSON = JavaScript Object Notation. Veri saklamak için kullanılan bir format. Python'daki dictionary'ye (sözlük) çok benzer.

```
# Python dictionary
kisi = {"isim": "Enes", "yas": 20}

# JSON (neredeyse aynı, dosyaya yazılmış hali)
{"isim": "Enes", "yas": 20}
```

GeoJSON = JSON + Coğrafi Bilgi

GeoJSON, JSON'un harita/konum bilgisi taşıyan özel bir versiyonu. Şartnameden gelen dosya şu yapıda:

```
{
  "type": "FeatureCollection",    <-- birden fazla nokta var
  "features": [                  <-- noktaların listesi
    {
      "type": "Feature",
      "properties": {
        "name": "start",          <-- noktanın ismi
        "description": "Başlangıç"
      },
      "geometry": {
        "type": "Point",
        "coordinates": [29.509, 40.790]
      }
    }
  ]
}
```

⚠ **GeoJSON'da koordinat sırası: [boylam, enlem] yani [longitude, latitude]. Google Maps'te ise enlem, boylam görürsün. TERS! Script bunu otomatik hallediyor.**

Dosyadaki Nokta Tipleri

İsim	Ne Demek	Yarışmadaki Anlamı
start	Başlangıç	Araç buradan harekete başlıyor
gorev_1	1. Görev	Yolcu BİNDİRME noktası (Rnd 3)
gorev_2	2. Görev	Yolcu İNDİRME noktası (Rnd 3)
gorev_3	3. Görev	Ek görev noktası (varsa)
park_giris	Park Girişi	Araç buraya park edecek

Bölüm 3: GPS Koordinatları ve Dönüşüm

GPS Ne Verir?

GPS sana enlem (latitude) ve boylam (longitude) verir. Bunlar DERECE cinsindendir:

Enlem: 40.790343° (kuzeye ne kadar yukarıda)

Boylam: 29.509014° (doğuya ne kadar sağda)

Robot Ne İstiyor?

Robotun navigasyon sistemi (Nav2) METRE cinsinden x, y koordinatı istiyor:

x = 0.0 metre (doğu-batı yönü)

y = 0.0 metre (kuzey-güney yönü)

Dönüşüm Nasıl Yapılıyor?

Dünya bir küre. Küre üzerindeki derece konumları, düz bir kağıt üzerindeki metre konumlara çevirmemiz lazım. Buna PROJEKSİYON denir. Biz UTM (Universal Transverse Mercator) projeksiyonu kullanıyoruz.

Mantık basit: Start noktasını (0,0) kabul et, diğer her noktanın start'a olan metre cinsinden uzaklığını hesapla.

GPS (derece)	UTM (metre)	Lokal (metre)
40.790343°N ----->	4516789m N ----->	0.000m (y)
29.509014°E ----->	327654m E ----->	0.000m (x)
40.789785°N ----->	4516727m N ----->	-62.93m (y)
29.508610°E ----->	327622m E ----->	-32.32m (x)

i pyproj kütüphanesi yoksa script Haversine formülüne geçer. Bu da yeterince doğru (5km'den az mesafelerde ~1cm hata).

Bölüm 4: Script Ne Yapıyor? (Parça Parça)

Script uzun görünüyor ama aslında 5 ana parçadan oluşuyor:

Parça 1: MissionPoint Sınıfı

Tek bir GPS noktasını tutan 'kutu'. GeoJSON'dan gelen bir feature alır, içinden isim, açıklama, koordinatları çıkarır.

```
class MissionPoint:
    self.name = 'gorev_1'      # noktanın ismi
    self.latitude = 40.789785  # enlem
    self.longitude = 29.508610 # boylam
    self.point_type = 'gorev'   # tip: start/gorev/park_giris
```

Parça 2: CoordinateConverter Sınıfı

GPS'ten metreye çeviren cetvel. Start noktasını (0,0) kabul edip diğer noktaların metre cinsinden yerini hesaplıyor.

Parça 3: GeoJSONMissionParser (Ana Beyin)

Tüm işi yapan ana sınıf:

- parse() → Dosyayı oku, JSON'u çöz
- _validate() → Doğrulama (start var mı? koordinat geçerli mi?)
- print_summary() → Terminal'e renkli güzel çıktı
- save_yaml() → YAML dosyası kaydet
- print_nav2() → C++ kodu üret
- print_raw_gps() → Düz GPS tablosu

Parça 4: Heading (Yön) Hesabı

Araç sadece NEREYE gideceğini değil, HANGİ YÖNE bakacağını da bilmeli. İki nokta arasındaki açı atan2(dy, dx) ile hesaplanır.

Robot bu açıyı QUATERNION formatında istiyor (4 sayı: x, y, z, w). Sadece yaw (sağa-sola dönüş) kullandığımız için:

```
qz = sin(açı / 2)
qw = cos(açı / 2)
qx = 0, qy = 0
```

i Quaternion'u şimdilik 'açıyı robotun anladığı formata çevirme' olarak düşün.

Parça 5: CLI (Komut Satırı Arayüzü)

argparse kütüphanesi ile yapılmış. Terminalde hangi bayrakları yazarsan ona göre çalışıyor:

```
python3 geojson_parser.py dosya.geojson      # özet
python3 geojson_parser.py dosya.geojson --yaml # YAML
```

```
python3 geojson_parser.py dosya.geojson --nav2    # C++ kodu
python3 geojson_parser.py dosya.geojson --raw-gps # tablo
python3 geojson_parser.py --sample                # örnek dosya
```

Bölüm 5: InitMissionAction ile Bağlantı

Bu en önemli kısım. Scriptimiz ile robotun beyni arasındaki bağlantı.

Robotun Beyni: Behavior Tree (BT)

Robotun ne yapacağını belirleyen bir karar ağacı var:

```
Robot açıldı
+-- Görev noktalarını yükle (InitMissionAction) <-- BİZİM İŞİMİZ
+-- Ana döngü başla
    |-- Engel var mı? --> Dur
    |-- Kırmızı ışık mı? --> Bekle
    |-- Tabela var mı? --> Uygula
    |-- Görev noktasına vardın mı? --> Yolcu al/bırak
+-- Hiçbiri yoksa --> Yoluna devam et
```

InitMissionAction Şu An Ne Yapıyor?

init_mission_action.cpp dosyasında şu an SAHTE (dummy) veriler var:

```
goal.pose.position.x = 0.0
goal.pose.position.y = 0.0
```

Bunları gerçek koordinatlarla değiştirmek lazım!

Akış Şeması

+-----+	+-----+	+-----+
GeoJSON Dosyası	---> geojson_parser.py	---> InitMissionAction
(hakemler verir)	(SENİN SCRIPTİN)	(C++ kodu)
+-----+	+-----+	+-----+
start: 40.79,29	GPS -> metre çevir	goals_list'e
gorev_1: ...	YAML veya C++ üret	waypoint ekle
park_giris: ...		BT'ye gönder
+-----+	+-----+	+-----+

Yarışma Günü Senaryosu

1. GeoJSON dosyasını al
2. python3 geojson_parser.py gorev.geojson --nav2 çalıştır
3. Çıkan C++ kodunu kopyala
4. init_mission_action.cpp'deki dummy kodu sil, yeni kodu yapıştır
5. colcon build ile derle
6. Aracı çalıştır

Bölüm 6: Test Etme Rehberi

Test 1: Scripti Çalıştır

```
cd ~/Documents/robotaxi_2026/src/robotaxi_bt/scripts
python3 geojson_parser.py --sample          # örnek dosya oluştur
python3 geojson_parser.py ornek_gorev.geojson # çalıştır
```

Beklenen: Renkli tablo, 5 nokta, mesafeler, açılar.

Test 2: YAML Kontrol

```
python3 geojson_parser.py ornek_gorev.geojson --yaml
cat ornek_gorev.yaml
```

Kontrol et:

- 5 waypoint var mı?
- start noktası $x=0$, $y=0$ mı? (orijin)
- latitude/longitude değerleri doğru mu?

Test 3: Kendi Dosyanı Yaz

Basit 2 noktalı bir test.geojson oluştur ve çalıştır. 0.001 derece fark $\sim$ 111 metre olmalı.

Test 4: Hata Durumları

```
echo "{}" > bozuk.geojson
python3 geojson_parser.py bozuk.geojson
# Beklenen: 'FeatureCollection' hatası
```

Test 5: Quaternion Doğrulama

Nav2 C++ çıktısındaki quaternion değerlerini kontrol et:

$$z^2 + w^2 = 1.0 \text{ olmalı}$$

$$\text{Örnek: } 0.525^2 + 0.851^2 = 0.276 + 0.724 = 1.0 \quad \checkmark$$

Bölüm 7: Yarışma Günü — Adım Adım

Yarışma Sabahı (Hazırlık)

```
cd ~/Documents/robotaxi_2026/src/robotaxi_bt/scripts
python3 geojson_parser.py --sample
python3 geojson_parser.py ornek_gorev.geojson
# Çalışıyorsa her şey tamam
```

Tur Öncesi (GeoJSON Geldiğinde)

1. Hakemlerden GeoJSON dosyasını al
2. scripts/ klasörüne kaydet
3. İlk bakış — kaç nokta, mesafeler, uyarılar:
python3 geojson_parser.py tur1_gorev.geojson
4. YAML ve C++ kodu üret:
python3 geojson_parser.py tur1_gorev.geojson --yaml --nav2
5. C++ kodunu init_mission_action.cpp'e yapıştır
6. colcon build → Aracı test et

Sorun Çıkarsa

Sorun	Çözüm
Koordinatlar yanlış	GeoJSON dosyasını tekrar kontrol et
Araç yanlış yere gidiyor	Datum noktasını kontrol et
start (0,0) değil	--datum-lat/lon ile belirt

Bölüm 8: Script Kod Haritası

Hangi satır aralığı ne yapıyor:

Satırlar	İçerik
1-50	Açıklama ve kullanım dokümanı
50-65	Kütüphane importları
68-75	ANSI renk kodları (terminal)
80-140	class MissionPoint (tek nokta)
145-195	class CoordinateConverter (GPS→metre)
200-440	class GeoJSONMissionParser (ana beyin)
445-510	create_sample_geojson() (örnek üretici)
515-590	main() ve CLI argüman işleme

Bölüm 9: Sık Sorulan Sorular

S: pyproj nedir, neden lazım?

C: GPS koordinatlarını metreye çevirmek için matematiksel bir kütüphane. Yoksa da çalışır (Haversine ile), ama UTM daha hassas.

S: Quaternion nedir?

C: 3 boyutlu uzayda bir dönüşü temsil eden 4 sayı (x,y,z,w). Biz sadece yaw (sağa-sola dönüş) kullandığımız için x=0, y=0. z ve w yeterli.

S: Datum ne demek?

C: Orijin noktası. 'Burayı (0,0) kabul et' dediğimiz yer. Varsayılan olarak 'start' noktası.

S: YAML nedir?

C: Konfigürasyon dosyası formatı. JSON'a benzer ama daha okunabilir. ROS2 dünyasında çok kullanılır.

S: Neden hem YAML hem C++ üretiyoruz?

C: YAML: C++ değiştirmeden yeni koordinat yüklenebilir. C++: Doğrudan yapıştırılabilir, en hızlı yol.

S: coordinates [29.x, 40.x] — hangi sıra?

C: GeoJSON standardı: [longitude, latitude] = [boylam, enlem]. Script bunu otomatik hallediyor.

Bölüm 10: Kaptana Sunum

Kaptanına şunu söyleyebilirsin:

"GeoJSON parser'ı yazdım. Yarışma günü dosyayı alıp tek komutla hem terminal özeti, hem YAML waypoints, hem de InitMissionAction için C++ kodu üretebiliyoruz.

GPS → UTM dönüşümü otomatik, heading quaternion hesabı var, doğrulama kontrolleri yapıyor (start var mı, koordinatlar Türkiye içinde mi, vs.).

Round 3 BT yapısıyla uyumlu:

start → gorev_1 → gorev_2 → park_giris sıralaması ve goal_type eşleştirmesi dahil."